

mgcal (minimal) — API Reference

A minimal C library for computing magnetic forward-modeling sensitivity matrices from rectangular-prism sources on a structured 3-D grid. This build is reduced to the **prism (3-D)** forward model and the `kernel_matrix_set` workflow; dipole, 2-D (yz), scattered, mapped, and per-row/column kernel routines have been removed.

All field values are multiplied by a global `scale_factor` (default `100.0`).

Conventions

- **Coordinate system:** z-axis points upward. The surface is at larger z, the subsurface at smaller (typically negative) z. A z-range such as `[0, -2]` runs from the surface down to 2 km depth, so cell thickness `dz` is stored as a **negative** value; the prism integrator compensates for the sign internally.
- **Memory ownership:** functions returning a pointer allocate it; the caller is responsible for freeing it with the matching `*_free` function (or `free()` for raw `double *` buffers).
- **Error handling:** invalid arguments and allocation failures call `error_and_exit` (from `util.h`), which prints a diagnostic and exits.
- **Angles:** inclination/declination are given in degrees.

Header layout

Include the umbrella header:

```
#include <mgcal.h>
```

It pulls in, in dependency order: `vector3d.h`, `data_array.h`, `grid.h`, `source.h`, `calc.h`, `io.h`, `kernel.h`. Individual headers are not self-contained and should not be included directly.

Data types

vector3d

Cartesian 3-vector.

```
struct s_vector3d { double x, y, z; };
```

data_array

Observation points and their measured values.

```
struct s_data_array {
    int      n;           // number of points
    double *x, *y, *z;    // coordinates
    double *data;         // observed value at each point
};
```

grid

Structured 3-D model grid ($n_x \times n_y \times n_z$ cells).

```
struct s_grid {
    int      n, nh, nx, ny, nz;    // total, horizontal-layer, and per-axis counts
    double xrange[2], yrange[2], zrange[2];
    double *x, *y, *z;            // cell-center coordinates per axis
    double *z1;                   // optional irregular surface (length nh)
    double *dx, *dy, *dz;         // per-axis cell sizes (dz may be negative)
    void *data;
};
```

source

A list of magnetized prism items plus an external-field direction. Built incrementally with `source_append_item`; the field callbacks iterate from `src->begin`.

mgcal_func

Binds a forward-model callback with an optional user parameter.

```
typedef double (*mgcal_theoretical)(const vector3d *pos, const source *src, void *data);
struct s_mgcal_func { mgcal_theoretical function; void *parameter; };
```

MgcalComponent

Selects which field component a callback returns.

```
typedef enum {
    MGCAL_X_COMPONENT,    // Hx
    MGCAL_Y_COMPONENT,    // Hy
    MGCAL_Z_COMPONENT,    // Hz
    MGCAL_TOTAL_FORCE     // total-force anomaly (exf . f)
} MgcalComponent;
```

vector3d.h — 3-vector primitives

Function	Summary
<code>vector3d *vector3d_new(double x, double y, double z)</code>	Allocate a vector with the given components.
<code>vector3d *vector3d_new_with_geodesic_poler(double r, double inc, double dec)</code>	Build a vector from magnitude and inclination/declination (degrees), geomagnetic convention: $x=r\cdot\cos(inc)\cdot\sin(dec)$, $y=r\cdot\cos(inc)\cdot\cos(dec)$, $z=-r\cdot\sin(inc)$.
<code>void vector3d_set(vector3d *cv, double x, double y, double z)</code>	Overwrite an existing vector's components in place.
<code>void vector3d_free(vector3d *cv)</code>	Free a vector (NULL-safe).
<code>void vector3d_scale(vector3d *x, double alpha)</code>	Multiply all components by <code>alpha</code> in place.
<code>vector3d *vector3d_copy(const vector3d *src)</code>	Allocate a copy of <code>src</code> .
<code>void vector3d_axpy(double alpha, const vector3d *x, vector3d *y)</code>	In-place $y = y + \alpha\cdot x$.
<code>double vector3d_dot(const vector3d *x, const vector3d *y)</code>	Dot product $x\cdot y$.
<code>double vector3d_nrm(const vector3d *c)</code>	Euclidean norm $ c $.

data_array.h — observation data

Function	Summary
<code>data_array *data_array_new(int n)</code>	Allocate a data array for <code>n</code> points; coordinate and value buffers are zero-initialized.
<code>void data_array_free(data_array *array)</code>	Free a data array and its buffers (NULL-safe).

grid.h — model grid

Function	Summary
<code>grid *grid_new(int nx, int ny, int nz, const double x[], const double y[], const double z[])</code>	Create a regular grid from corner ranges (each a <code>[min,max]</code> pair); cell sizes from even division. Caveat: for an axis with a single cell, the range's second value is ignored and that cell size becomes zero — use <code>grid_new_full</code> instead.
<code>grid *grid_new_full(int nx, int ny, int nz, const double x[], const double y[], const double z[], const double *dx, const double *dy, const double *dz, const double *z1)</code>	Create a grid with explicit per-axis cell sizes and an optional irregular surface <code>z1</code> (length <code>nx·ny</code>). Pass NULL for any of <code>dx/dy/dz</code> to fall back to even division.
<code>bool grid_set_surface(grid *g, const double *z1)</code>	Attach or replace an irregular top surface (<code>z1</code> , length <code>nx·ny</code>); each cell center's <code>z</code> is offset by <code>z1[h]</code> when queried. Frees any previous surface. Returns true on success.
<code>void grid_free(grid *g)</code>	Free a grid and all coordinate/spacing arrays (NULL-safe).
<code>void grid_get_index(const grid *g, int n, int *i, int *j, int *k, int *h)</code>	Decompose flat cell index <code>n</code> into <code>(i,j,k)</code> and horizontal index <code>h</code> ; any output pointer may be NULL.
<code>void grid_get_nth(const grid *g, int n, vector3d *center, vector3d *dim)</code>	Get the center coordinate and/or cell dimension of cell <code>n</code> (center includes the <code>z1</code> offset when present); either output may be NULL.
<code>void grid_stretch_at_edge(grid *g, double l)</code>	Extend the outermost cells outward by length <code>l</code> without adding cells: horizontal edges (<code>x, y</code>) grow on both sides, and the deepest <code>z</code> -layer is extended downward (consistent with <code>z</code> -up, negative <code>dz</code>).

source.h — magnetized sources

Function	Summary
<code>source *source_new(void)</code>	Create an empty source (no external field, no items).
<code>void source_free(source *src)</code>	Free a source, its item list, and the external field.
<code>void source_set_external(source *src, double inc, double dec)</code>	Set the external-field direction from inclination/declination (degrees) as a unit vector.
<code>int source_append_item(source *src)</code>	Append a new empty magnetized item; sets <code>src->begin</code> on first append, updates <code>src->end</code> , returns the item index. Fill <code>pos / dim / mgz</code> directly afterward.

calc.h — prism forward model

Function	Summary
<code>vector3d *prism(const vector3d *obs, const source *s)</code>	Total magnetic field at <code>obs</code> from all prism items in <code>s</code> ; sums the signed 8-vertex contributions and applies <code>scale_factor</code> . Returns a newly allocated vector (caller frees).
<code>double total_force(const vector3d *exf, const vector3d *f)</code>	Total-force anomaly: projection $\text{exf} \cdot \text{f}$ of field <code>f</code> onto the external-field direction.
<code>double x_component_prism(const vector3d *obs, const source *src, void *data)</code>	<code>mgcal_func</code> callback: x-component of the prism field.
<code>double y_component_prism(const vector3d *obs, const source *src, void *data)</code>	<code>mgcal_func</code> callback: y-component of the prism field.
<code>double z_component_prism(const vector3d *obs, const source *src, void *data)</code>	<code>mgcal_func</code> callback: z-component of the prism field.
<code>double total_force_prism(const vector3d *obs, const source *src, void *data)</code>	<code>mgcal_func</code> callback: total-force anomaly of the prism field.

The four `*_component_*` callbacks all match the `mgcal_theoretical` signature and are intended to be passed to `mgcal_func_new`. The `data` argument is unused.

Singularity handling: when an observation point coincides with a prism vertex ($r = 0$), the kernel returns zero contribution instead of producing `NaN`.

kernel.h — sensitivity matrix assembly

Function	Summary
<code>mgcal_func *mgcal_func_new(mgcal_theoretical func, void *data)</code>	Bundle a forward-model callback with a user parameter. Returns a new <code>mgcal_func</code> . Ownership of <code>data</code> stays with the caller.
<code>void mgcal_func_free(mgcal_func *f)</code>	Free an <code>mgcal_func</code> (does not free the user parameter).
<code>void kernel_matrix_set(double *a, const data_array *array, const grid *g, const vector3d *mgz, const vector3d *exf, const mgcal_func *f)</code>	Fill a preallocated sensitivity matrix. Computes $a[l + j \cdot m] = f(\text{obs}_l, \text{cell}_j)$ for all <code>m</code> observations and all <code>g->n</code> cells, with fixed magnetization <code>mgz</code> and external field <code>exf</code> . Column-major in the cell index. OpenMP-parallel over z-layers. Caller allocates <code>m · g->n</code> doubles.
<code>double *kernel_matrix(const data_array *array, const grid *g, const vector3d *mgz, const vector3d *exf, const mgcal_func *f)</code>	Convenience wrapper: allocates <code>m · g->n</code> doubles, calls <code>kernel_matrix_set</code> , and returns the buffer (caller frees with <code>free()</code>).

Matrix layout

For `m` observation points and `g->n` grid cells, the output buffer holds the matrix in cell-major order: cell `j` occupies the contiguous block `a[j·m .. j·m + m - 1]`, where entry `l` within that block is the response of observation `l` to a unit source in cell `j`. This corresponds to a column of the sensitivity matrix `G` (observations × cells).

io.h — file input / output

Function	Summary
<code>data_array *fread_data_array(FILE *stream)</code>	Read observation data (<code>x y z</code> value per line) from a stream; skips comment (<code>#</code>) and malformed lines. Returns a new <code>data_array</code> .
<code>void fwrite_data_array_with_data(FILE *stream, const data_array *array, const double *data, const char *format)</code>	Write <code>x/y/z</code> from <code>array</code> paired with an external <code>data</code> vector, one record per line, using <code>format</code> (4 fields).
<code>void fwrite_data_array(FILE *stream, const data_array *array, const char *format)</code>	Write a data array using its own stored values.
<code>grid *fread_grid(FILE *stream)</code>	Parse a grid definition (dimensions, corner positions, per-axis coordinate and spacing arrays, optional <code>z1</code>) from a stream. Returns a new <code>grid</code> .

<code>void fwrite_grid(FILE *stream, const grid *g)</code>	Write a grid definition in the format readable by <code>fread_grid</code> .
<code>void fwrite_grid_to_xyz(FILE *stream, const grid *g, const char *format)</code>	Write the center coordinate of every cell as <code>x y z</code> .
<code>void fwrite_grid_with_data(FILE *stream, const grid *g, const double *data, const char *format)</code>	Write each cell center paired with a per-cell value (<code>x y z value</code>); NULL <code>data</code> writes zeros. Typically used to export inversion models.

mgcal.h — global scale factor

Function	Summary
<code>void mgcal_set_scale_factor(double val)</code>	Set the global output scale applied to all field values.
<code>double mgcal_get_scale_factor(void)</code>	Return the current global scale factor.
<code>extern double scale_factor</code>	The global scale (default <code>100.0</code>); defined in <code>calc.c</code> .

Typical workflow

```
#include <mgcal.h>

/* 1. Build the model grid (explicit dz, negative for downward). */
double x[2] = {0.0, 0.0}, y[2] = {0.0, 0.0}, z[2] = {0.0, 0.0};
double dx[1] = {1.0}, dy[1] = {1.0}, dz[1] = {-1.0};
grid *g = grid_new_full(1, 1, 1, x, y, z, dx, dy, dz, NULL);

/* 2. Load observations. */
data_array *obs = fread_data_array(fp);

/* 3. Choose magnetization and external field (e.g. inc=45, dec=0). */
vector3d *mgz = vector3d_new_with_geodesic_poler(1.0, 45.0, 0.0);
vector3d *exf = vector3d_new_with_geodesic_poler(1.0, 45.0, 0.0);

/* 4. Wrap the desired forward model (total-force here). */
mgcal_func *f = mgcal_func_new(total_force_prism, NULL);

/* 5. Build the sensitivity matrix G (obs x cells), cell-major. */
double *G = kernel_matrix(obs, g, mgz, exf, f);

/* ... use G in the inversion ... */

free(G);
mgcal_func_free(f);
vector3d_free(mgz);
vector3d_free(exf);
data_array_free(obs);
grid_free(g);
```

Internal utilities

The sources depend on `src/util.h` / `src/util.c` for `error_and_exit`, `array_copy`, `array_set_all`, and `set_range`. These are internal helpers, not part of the public API, and `util.h` is not installed alongside the public headers.